

Paginação com envelope — o caminho B

O formato que eu recomendo como padrão para o mercado — LogisticaAPI, kata 5

Você já tem o caminho A quase pronto. Este guia é a conversão: o que sai, o que entra e o que não muda. No fim, um upgrade que só vale a pena depois que o envelope está no lugar — projetar o DTO direto no SQL, que elimina o Include e o mapeamento em memória de uma vez só.

1. O que muda

Menos do que parece. A query não muda em nada — a diferença toda está no tipo que atravessa as camadas e em como ele chega ao JSON.

Arquivo	O que acontece
Paginacao/QueryableParameters.cs	não muda — a entrada é a mesma nos dois caminhos
Paginacao/PagedList.cs	sai; vira PagedResult<T> por composição
Paginacao/MetaData.cs	não precisa mais — os metadados voltam para dentro do envelope
Paginacao/QueryableExtensions.cs	entra: o helper genérico devolvendo PagedResult
PedidoRepository.GetPaged	só muda o tipo de retorno; a query é idêntica
IPedidoService / PedidoService	só muda o tipo de retorno
PedidoController.GetAll	sai o header X-Pagination, entra o Map

O MetaData separado deixa de fazer sentido — e isso é informação

No caminho A ele existia por uma razão concreta: dar um objeto pronto para serializar no header em uma linha. No envelope não há header, e os metadados já estão no mesmo objeto que os itens. Manter a classe separada aqui seria carregar uma indireção sem motivo. Repare no padrão: cada peça daquele desenho existia por causa da *consequência* de herdar de `List<T>`. Trocada a decisão de base, metade das peças some.

2. PagedResult<T> — composição no lugar de herança

```
// LogisticaAPI/Paginacao/PagedResult.cs
namespace LogisticaAPI.Paginacao;

public class PagedResult<T>
{
    public IReadOnlyList<T> Items { get; init; } = [];

    public int PageNumber { get; init; }
    public int PageSize { get; init; }
    public int TotalCount { get; init; }

    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
    public bool HasPrevious => PageNumber > 1;
    public bool HasNext => PageNumber < TotalPages;

    // converte a pagina de entidade para a pagina de DTO,
    // preservando os metadados sem repeti-los em cada controller
    public PagedResult<TDestino> Map<TDestino>(Func<T, TDestino> seletor) => new()
    {
        Items = Items.Select(seletor).ToList(),
        PageNumber = PageNumber,
        PageSize = PageSize,
        TotalCount = TotalCount
    };
}
```

- **Items é propriedade, não herança.** É a mudança de fundo: o tipo não implementa IEnumerable, então o System.Text.Json o trata como objeto comum e serializa todas as propriedades — que era exatamente o que se perdia no caminho A.
- **init em vez de set.** As propriedades só podem ser atribuídas na criação do objeto. Uma página é um retrato de um instante; ninguém deveria alterar o TotalCount depois que ela foi montada. O compilador passa a garantir isso.
- **TotalPages continua derivado.** Não se guarda o que dá para calcular — guardar abre a porta para o valor ficar dessincronizado do TotalCount. E o (double) continua obrigatório pelo mesmo motivo de sempre: sem ele, 101 itens em páginas de 20 viram 5 páginas.
- **IReadOnlyList<T>.** Comunica no tipo que a lista é para leitura. Quem receber a página não tem Add nem Remove à disposição — e não deveria mesmo ter.

O Map é a peça que evita a repetição chata

Sem ele, todo controller teria que copiar os quatro campos de metadado na mão, e um PageSize = pagina.PageNumber distraído passaria despercebido. Com ele, a conversão de página de entidade para página de DTO vira uma linha, em qualquer controller, para qualquer entidade. É o mesmo raciocínio do helper genérico: o que é mecânico e se repete entre entidades mora em um lugar só.

3. O helper genérico

Arquivo novo

```
// LogisticaAPI/Paginacao/QueryableExtensions.cs
using Microsoft.EntityFrameworkCore;

namespace LogisticaAPI.Paginacao;

public static class QueryableExtensions
{
    public static async Task<PagedResult<T>> ToPagedResultAsync<T>(
        this IQueryable<T> source, QueryableParameters parametros)
    {
        var total = await source.CountAsync();

        var items = await source
            .Skip((parametros.PageNumber - 1) * parametros.PageSize)
            .Take(parametros.PageSize)
            .ToListAsync();

        return new PagedResult<T>
        {
            Items = items,
            PageNumber = parametros.PageNumber,
            PageSize = parametros.PageSize,
            TotalCount = total
        };
    }
}
```

É o seu `ToPagedListAsync` com duas diferenças: virou *extension method* (a chamada continua a cadeia LINQ em vez de quebrá-la) e recebe o `QueryableParameters` inteiro em vez de dois inteiros soltos — assim não existe a chance de trocar a ordem dos argumentos e paginar com `(pageSize, pageNumber)`, erro que o compilador não pega porque os dois são `int`.

4. Repositório, Service e Controller

Repositório — a query é a mesma do caminho A

```
// LogisticaAPI/Repositories/IPedidoRepository.cs
Task<PagedResult<Pedido>> GetPaged(QueryableParameters parametros);

// LogisticaAPI/Repositories/PedidoRepository.cs
public async Task<PagedResult<Pedido>> GetPaged(QueryableParameters parametros)
{
    return await _dbContext.Pedidos
        .AsNoTracking()
        .Include(p => p.ItensPedido)
        .AsSplitQuery()
        .OrderByDescending(p => p.CriadoEm)
        .ThenBy(p => p.PedidoId)
        .ToPagedResultAsync(parametros);
}
```

Service — repasse puro

```
// LogisticaAPI/Services/IPedidoService.cs
Task<PagedResult<Pedido>> GetPaged(QueryableParameters parametros);

// LogisticaAPI/Services/PedidoService.cs
public async Task<PagedResult<Pedido>> GetPaged(QueryableParameters parametros)
{
    return await _pedidoRepository.GetPaged(parametros);
}
```

Controller — some o header, some o JsonSerializer, some o using System.Text.Json

```
// LogisticaAPI/Controller/PedidoController.cs
[HttpGet]
[ProducesResponseType(typeof(PagedResult<PedidoResponseDto>), StatusCodes.Status200OK)]
public async Task<ActionResult<PagedResult<PedidoResponseDto>>> GetAll(
    [FromQuery] QueryableParameters parametros)
{
    var pagina = await _pedidoService.GetPaged(parametros);
    return Ok(pagina.Map(MapToResponse));
}
```

Três linhas. O `MapToResponse` que você já tinha continua servindo sem alteração — o `Map` recebe justamente um `Func<Pedido, PedidoResponseDto>`, que é a assinatura dele.

O `[ProducesResponseType]` não é decoração: é o que faz o Swagger mostrar o schema com `totalCount` e `hasNext` em vez de um object genérico. Como o retorno já é `ActionResult<PagedResult<...>>`, o Swashbuckle infere boa parte sozinho, mas o atributo deixa explícito e cobre o caso de você passar a devolver `IActionResult` depois.

O JSON resultante

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "items": [
    { "pedidoId": 11, "cliente": "ACME", "itens": [ ... ] },
    { "pedidoId": 12, "cliente": "Bosch", "itens": [ ... ] }
  ],
  "pageNumber": 2,
  "pageSize": 10,
  "totalCount": 63,
  "totalPages": 7,
  "hasPrevious": true,
  "hasNext": true
}
```

Isto é uma quebra de contrato — e é a única desvantagem real do caminho B

O corpo deixa de ser array e vira objeto. Todo cliente existente para de funcionar, sem erro de compilação, sem aviso — só um `undefined` no front. Aqui não há consumidor, então é indolor. Em produção, isso exigiria versionar a API (`/api/v2/pedidos`) e manter as duas durante uma janela de migração. Saber nomear isso vale mais em entrevista do que a implementação em si.

5. O upgrade: projetar o DTO direto no SQL

Com o envelope no lugar, dá para dar um passo que melhora de verdade a consulta. Hoje o fluxo é: trazer as entidades completas → montar objetos C# → mapear para DTO → descartar as entidades. O banco manda colunas que ninguém usa.

Como o helper é genérico em T (e não em uma entidade), ele aceita uma query **já projetada**:

```
// LogisticaAPI/Repositories/PedidoRepository.cs
public async Task<PagedResult<PedidoResponseDto>> GetPagedProjetado(
    QueryableParameters parametros)
{
    return await _dbContext.Pedidos
        .AsNoTracking()
        .OrderByDescending(p => p.CriadoEm)
        .ThenBy(p => p.PedidoId)
        .Select(p => new PedidoResponseDto
        {
            PedidoId = p.PedidoId,
            TipoPedido = p.TipoPedido,
            Cliente = p.Cliente,
            CriadoEm = p.CriadoEm,
            Itens = p.ItensPedido.Select(i => new ItemPedidoResponseDto
            {
                ItemPedidoId = i.ItemPedidoId,
                ItemId = i.ItemId,
                Descricao = i.Descricao,
                Quantidade = i.Quantidade,
                PesoUnitario = i.PesoUnitario,
                VolumeUnitario = i.VolumeUnitario,
                AlturaUnitario = i.AlturaUnitario,
                PesoTotal = i.PesoUnitario * i.Quantidade,
                VolumeTotal = i.VolumeUnitario * i.Quantidade
            }).ToList()
        })
        .ToPagedResultAsync(parametros);
}
```

- O **Include desaparece**. Não é esquecimento: o `Select` aninhado já diz ao EF quais dados dos itens ele precisa. `Include` serve para carregar navegações de *entidades*; em projeção, ele é ignorado.
- O `AsSplitQuery` **também sai** — sem `Include` de coleção não há explosão cartesiana para mitigar.
- **Cálculos vão para o banco**. `PesoUnitario * Quantidade` vira multiplicação no SQL. Some o mapeamento em memória do controller.
- **Menos colunas na rede**. Só o que o DTO declara trafega, não a entidade inteira.

A armadilha da projeção

Dentro de um `Select` sobre `IQueryable` você **não pode chamar o seu** `MapToResponse`. `.Select(MapToResponse)` compila — e estoura em tempo de execução com *"could not be translated"*, porque o EF precisa ler a árvore de expressão para gerar SQL, e um método compilado é uma caixa-preta para ele. A expressão tem que estar inline, como acima. É o preço da projeção: o mapeamento fica duplicado entre o método e a query.

Por isso este é um **upgrade opcional**, e não o padrão do guia: ele troca duplicação de mapeamento por eficiência. Vale quando a listagem é quente e o DTO é bem menor que a entidade. Faça a versão simples funcionar primeiro; converta depois, se quiser sentir a diferença no SQL logado.

6. Checklist

Teste	Esperado
GET ?pageNumber=1&pageSize=5	objeto com itens de 5 pedidos, cada um com itens preenchidos
olhar o corpo, não o header	totalCount, totalPages e hasNext presentes no JSON
abrir o Swagger	schema mostrando os campos do envelope, não um array
ler o SQL logado	contém LIMIT
?pageSize=999999	no máximo 50 itens
?pageNumber=0	primeira página, sem 500
?pageNumber=999	200 com itens vazio e hasNext false — não 404
última página	hasNext false, hasPrevious true
101 registros, pageSize 20	totalPages = 6, não 5

Os dois últimos são os que pegam o erro de divisão inteira e o de página fora da faixa — justamente os que passam despercebidos quando se testa só a página 1 com poucos registros.

7. Roteiro de conversão, na ordem

- Criar `Paginacao/PagedResult.cs` e `Paginacao/QueryableExtensions.cs`.
- Apagar `PagedList.cs` (e `Metadata.cs`, se você chegou a criar).
- Ajustar o namespace: a pasta é `LogisticaAPI/Paginacao/`, então o namespace é `LogisticaAPI.Paginacao` — não `LogisticaAPI.DTOS.Paginacao`, que era o desencontro que o Rider estava sublinhando.
- Trocar o tipo de retorno em `IPedidoRepository` → `PedidoRepository` → `IPedidoService` → `PedidoService` → `PedidoController`, nessa ordem, deixando o compilador guiar.
- No controller: apagar o `Response.Headers.Append`, o `using System.Text.Json`, e usar `pagina.Map(MapToResponse)`.
- Rodar o checklist da seção 6.
- Replicar em `Item` e `Carregamento` — sem `Include`, logo sem `AsSplitQuery`. `TipoPaleta` fica de fora.

Vale guardar o caminho A em um branch

Você tem duas implementações do mesmo problema, com trade-offs opostos e ambas suas. Isso é material de entrevista melhor do que qualquer certificado: dá para contar a decisão, não só o resultado. Um `git branch paginacao-header` antes de converter preserva o A sem poluir a main.